

AI-Powered Presentation Generation

A design-system-first architecture for generating decks on par with Canva, Gamma, and Beautiful.ai

Version 1 · July 17, 2026 · Owner: Product

1. Summary

We are building an AI-native presentation design tool — think "**Canva meets Gamma**" — where a user gives a prompt, outline, or document, and the system generates a complete, polished, on-brand slide deck: content, imagery, layout, and design, editable afterward in a full drag-and-drop canvas.

Core bet: market leaders (Gamma, Beautiful.ai, Tome, Canva Magic Design) win not because their AI is smarter, but because they tightly constrain what the AI is allowed to decide, and invest heavily in a deterministic design system underneath it. We follow the same architecture.

2. Problem Statement

Users who need decks fast face a tradeoff today between **manual tools** (full control, slow, requires design skill) and **naive AI generators** (fast, but generic, text-heavy, and visually inconsistent). Nobody has fully solved "fast AND good-looking AND editable" for our target segment.

3. Goals

PRODUCT GOALS

- Generate a full 10–20 slide deck in under 60 seconds with zero manual layout work required
- Output visually on par with human-designed Canva/Gamma decks
- Every slide fully editable afterward in a Canva-style canvas
- Export to PPTX, PDF, and shareable web link

NON-GOALS (V1)

- Real-time multiplayer co-editing
- Video / motion slides
- Custom animation / transition builder
- Enterprise brand-kit governance

4. Target Users

Persona	Use Case	What "Good" Looks Like
Founder / sales rep	Pitch deck, sales deck	Clean, credible, investor/client-ready in one pass
Consultant / PM	Client report, project update	Data-dense but readable, charts render well
Student / educator	Lecture deck, project presentation	Clear structure, engaging visuals, fast to produce
Marketer	Product launch, webinar deck	On-brand, visually distinctive, image-forward

5. Competitive Landscape

Tool	Strength	Weakness We Exploit
Gamma	Best-in-class AI generation quality	Limited manual design control, generic at scale
Canva Magic Design	Huge template/asset library	Weaker AI generation step, needs manual cleanup
Beautiful.ai	Strong smart-template auto-layout	Dated visual style, smaller template variety
Tome	Strong narrative/AI-first UX	Less mature manual editing, inconsistent exports
PowerPoint Copilot	Deep Office integration	Weak design taste, requires PowerPoint license

6. Core Architecture

This is the crux of the PRD: the AI is **never** asked to freehand a layout. It classifies and selects; a deterministic design system renders.

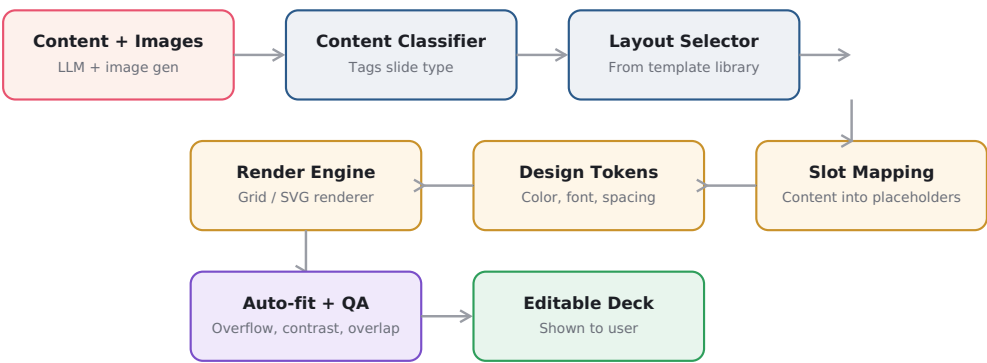


Fig. 1 — End-to-end generation pipeline

6.1 Content Generation

- LLM generates the outline (slide count, titles, narrative arc) first — confirmed/editable by the user before full generation
- Per-slide content is capped by hard limits (max words, max bullets), enforced at generation time. Overflow triggers slide-splitting, not font-shrinking

6.2 Content Classifier

Each slide is tagged with a content-type label: title, agenda, bullet list, comparison, stat/big-number, quote, image+text, timeline, team/grid, chart/data, closing, section-divider. A structured-output LLM call — cheap and highly reliable.

6.3 Layout Template Library

40–60 hand-designed base layouts across the content-type taxonomy (5–8 variants per type), each defined as a JSON layout schema, not an image.

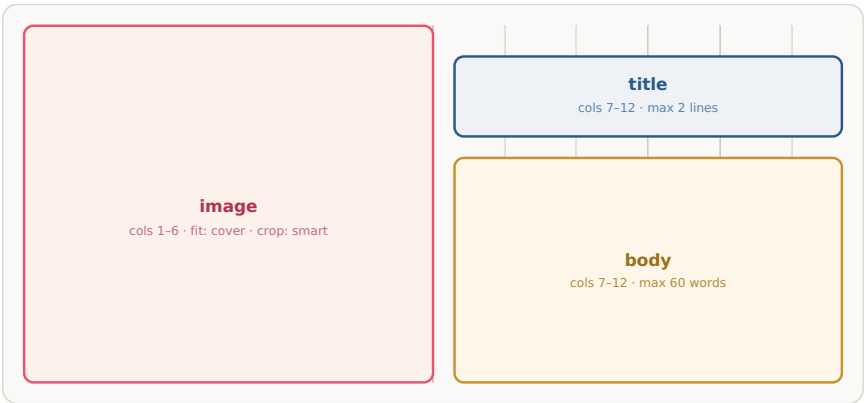


Fig. 2 — Layout defined as a grid schema, not pixels

```
{
  "layout_id": "image_text_split_v2",
  "content_type": "image+text",
  "grid": "12-col",
  "slots": [
    { "id": "image", "region": "cols 1-6, rows 1-12", "fit": "cover", "crop": "smart" },
    { "id": "title", "region": "cols 7-12, rows 2-4", "max_lines": 2 },
    { "id": "body", "region": "cols 7-12, rows 5-10", "max_words": 60 }
  ]
}
```

6.4 Layout Selector

Rule-based first (deterministic, debuggable). Inputs: content-type tag, word count, bullet count, image aspect ratio, presence of chart/table data. Avoids repeating the same variant on consecutive same-type slides.

6.6 Design Tokens (Theming)

A single theme object per deck: color palette (contrast-checked programmatically, never freeform LLM hex), font pairing (from 10-15 curated pairings), fixed type scale, spacing scale, image treatment — applied uniformly by the renderer everywhere.

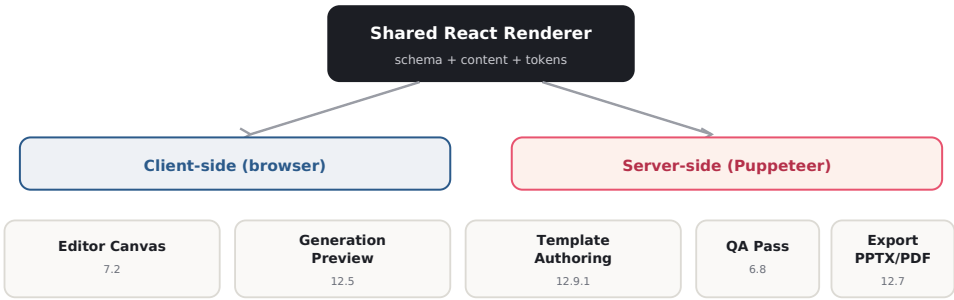
6.7 Render Engine

A deterministic CSS-grid/SVG renderer draws {schema + content + tokens}. All precise coordinate/pixel placement lives here — never in the LLM. The same renderer powers both the generated deck and the manual editor, guaranteeing what-you-generate-is-what-you-edit consistency.

6.7.1 ONE RENDERER, FIVE CONSUMERS

The render engine is not a single-purpose export tool — it is the most reused piece of infrastructure in the system. Five distinct parts of the product depend on it producing an identical result from the same schema:

Consumer	What It Does	Where Else Mentioned
Live editor canvas	Renders each slide as the user drags/resizes/edits it	7.2
In-app preview during generation	Renders slides progressively as they complete	12.5
Auto-fit / QA validation pass	Renders each schema against real content to check overflow, contrast, overlap	6.8
Template authoring preview	Renders live as an admin drags a slot on the grid picker	12.9.1
Server-side export (PPTX/PDF)	Renders each slide for conversion to PPTX objects / PDF pages	12.7



Same component code, two execution contexts — pixel-identical output by construction

Fig. 8 — One renderer, five consumers, two execution contexts

If each of these five ran its own separate rendering logic, they would drift out of sync — a deck could look right in the editor and break on export. The WYSIWYG guarantee in 12.7 is only true if this is guaranteed by construction, not by convention.

Architecture decision: one shared React renderer, two execution contexts. The render engine is a single schema-driven React component package, consumed two ways — **client-side**, directly, inside the editor UI, generation-progress preview, and template-authoring grid picker; and **server-side**, via Puppeteer navigating to an internal render-only route that mounts the identical component package given a schema/content/tokens payload, used for the QA validation pass and export. Because both contexts run the same component code — not a reimplementation, not a parallel template layer — pixel-identical output is structural, not something manually kept in sync. This mirrors Remotion's existing pattern in the video pipeline (React components rendered server-side for final export), applied to decks instead of scenes.

6.8 Auto-fit & QA Validation Pass

Before the user sees output, an automated pass checks: text overflow, contrast (WCAG), element overlap, whitespace balance — auto-correcting or falling back to a simpler layout variant rather than shipping a broken render.

6.5 Image Strategy

Image selection is decoupled from slide copywriting and run as its own structured step — the single biggest fix for generic, cliché stock imagery in AI-generated decks.

Sources are first-class and chosen per slide, not defaulted to stock: stock library, AI-generated photo/illustration, render-engine-built charts/diagrams/icons, and Path B composite images. A deck generated end-to-end by AI needs every one of these — a real deck mixes photos, icons, data charts, and the occasional bespoke diagram, and each needs the right source, not a single fallback path.

FULL VISUAL-TYPE ROUTING

Visual Need	Example	Source
Relatable photo/scene	"warehouse automation," "remote team meeting"	Stock search or AI photo generation, chosen per slide — not stock-first
Relatable illustration	Abstract-but-concrete concept needing a drawn style	AI illustration generation, styled via locked image_style token
Icon	Single glyph for a bullet/label	Icon library (render engine) — never image search/gen

Chart / data visualization	Bar chart, pie chart, stat callout	Render engine, built from structured numeric data — never image search/gen
Simple structured diagram	3-step process, comparison, timeline	Render engine template (6.3) — editable, on-brand
Bespoke composite diagram/infographic	Custom architecture, database schema, multi-panel infographic	Path B — AI image gen with baked-in text/layout (6.5.2)

This table is what the content classifier + image-brief step (6.5.1) actually decides against — for every slide, the correct row is picked based on content type and structure, not a single default.

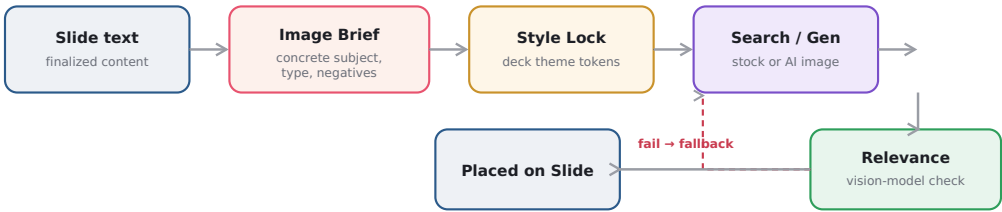


Fig. 3 — Image brief generation and validation pipeline

IMAGE BRIEF PROMPT

You are selecting the visual for a single presentation slide.
Output a JSON image brief - describe a concrete, literal visual,
not an abstract theme.

Rules:

- Extract 1-3 CONCRETE nouns/entities from the slide content -
never abstract concepts like "growth" or "success" alone.

- Avoid cliché stock tropes: handshakes, lightbulbs, people
pointing at whiteboards, puzzle pieces, rocket ships.

- Prefer specificity: "server rack in a dim data center" beats
"technology background."

- Flag image_type as diagram/chart (not photo) when the slide
needs to convey actual information or a relationship.

Output schema:

{

"subject": "...", "image_type": "photo|illustration|diagram|chart|icon",

"composition": "...", "search_query": "...",

"negative_terms": [...], "alt_text": "..."

}

STYLE LOCKING

Deck-wide visual style tokens are appended to every generated brief so subject matter can vary while treatment never does:

```
final_prompt = f"{brief.search_query}, {deck_theme.image_style}, {deck_theme.color_treatment}"
```

RELEVANCE VALIDATION

A vision-model check compares the candidate image against the slide's actual text before placement. A fail triggers fallback to the next candidate or regeneration — catching results that are topically adjacent but not truly relevant.

6.5.2 Composite Infographics & Technical Diagrams (Path B)

Bespoke, one-off technical content — system architecture diagrams, database ERDs, multi-panel process infographics — can't realistically be templated for every structure a user might need. For this class of content, a second path is used: a text-rendering-capable image model (e.g. GPT-Image, Ideogram) generates the entire diagram as one composite image, with boxes, arrows, icons, and text baked in by the model itself.

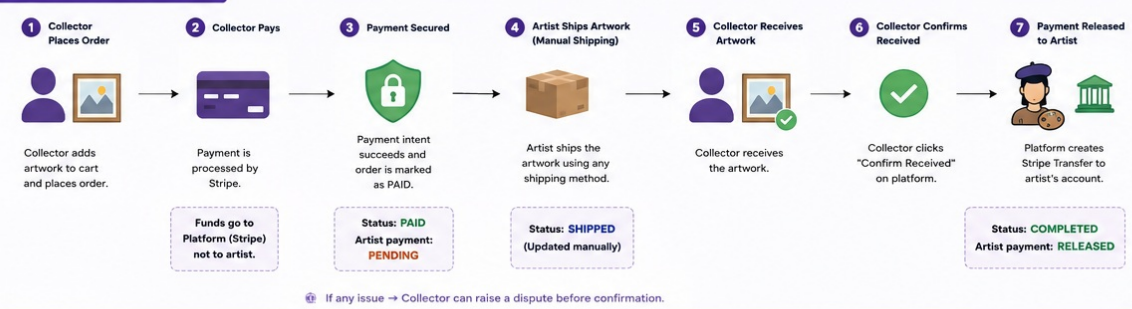
Benchmark reference (Fig. 5): an internally reviewed multi-panel payment-flow / architecture / ERD infographic sets the quality bar for this path — shared visual language across panels, a small consistent color-coding legend, sparing flat iconography, and structured data (schema fields, API endpoints, states) rendered as real layout, not prose.

Art Marketplace – Stripe Payment Flow (Initial Phase)

(Collector Pays → Released to Artist After Confirmation)

Easyship integration is NOT included in the initial phase.

1. PAYMENT FLOW (HIGH LEVEL)



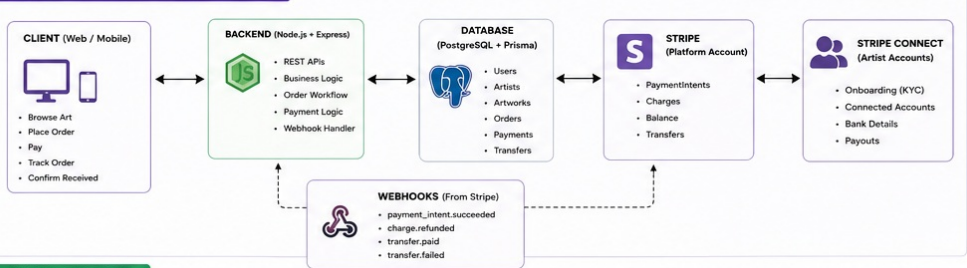
Key Points (Initial Phase)

- ✓ Payments are processed by Stripe.
- ✓ Funds are held with the platform (not in escrow).
- ✓ Artist gets paid only after collector confirms receipt.
- ✓ Manual shipping (no Easyship).
- ✓ Platform can refund if needed before confirmation.

Out of Scope (Phase 1)

- ✗ Easyship integration
- ✗ Auto-release after delivery
- ✗ Shipping tracking sync
- ✗ Dispute automation
- ✗ Advanced escrow features

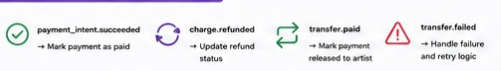
2. SYSTEM ARCHITECTURE (INITIAL PHASE)



4. KEY API ENDPOINTS



5. WEBHOOK EVENTS (FROM STRIPE)



6. ORDER STATES (FLOW)



7. IMPORTANT BUSINESS RULES (INITIAL PHASE)

- Payment captured by Stripe when collector pays.
- Platform holds the funds (not artist).
- Artist gets paid only after collector confirms receipt.
- Refunds are processed by platform before confirmation.
- Stripe Connect required for artists to receive payouts.
- All monetary amounts are in smallest currency unit (e.g. cents).

Fig. 5 — Benchmark composite infographic (Path B quality bar)

Content Brief

Path A · Render Engine

- Matches an existing template
- Fully editable, pixel-consistent
- Guaranteed accurate text/data
- Best for: bullets, comparisons, timelines, stat cards

Path B · AI Image Gen

- Bespoke, one-off structure
- Not editable, needs review
- Text-rendering-capable model
- Best for: architecture diagrams, schemas, multi-panel infographics

Fig. 6 — Path A vs. Path B routing

Routing rule: Path A is preferred whenever content fits an existing (or reasonably templatable) diagram type, since it stays editable and pixel-consistent with the rest of the deck. Path B is used only when content is genuinely bespoke and building a reusable template isn't practical.

Boundary clarification: this is not "Path A = stock, Path B = AI generation" — both paths can use AI image generation. The distinction is what's being generated: Path A produces a single-subject image (one photo, illustration, or icon) alongside real, separately-editable slide text. Path B produces a whole flattened composition where text and structure are baked into the pixels because no template can represent that arbitrary structure. Defaulting everything to Path B would work technically but costs editability, deck-wide visual consistency (Path B isn't built from locked design tokens), and cost (priced higher in 12.10 since it does more per call).

PATH B PROMPT PATTERN

Create a clean, professional infographic-style diagram, [N] panels arranged in a grid, white background, [brand color palette] accent colors used consistently (specify what each color means).

Panel 1 ("[title]"): [exact flow/steps with labels, in order]

Panel 2 ("[title]"): [exact boxes/fields, in order]

...

Style: flat icons, rounded rectangle boxes, thin arrows connecting steps, numbered circular badges for sequence, consistent sans-serif

typography, no photorealistic elements.

Content itself (panel titles, exact step labels, field names, color-legend meaning, panel order) is fully determined upstream by content generation — the image model's job is purely visual typesetting of a spec it's given, the same "classify and select, don't freehand" principle applied to a different renderer.

7. Functional Requirements

7.1 GENERATION FLOW

ID	Requirement
FR1	Start from prompt, pasted outline, or uploaded document (PDF/DOCX)
FR2	Editable outline shown before full generation
FR3	Theme selected or auto-assigned before generation
FR4	Full 15-slide deck generates in under 60 seconds
FR5	Regenerate a single slide/element without losing manual edits elsewhere

7.2 EDITING CANVAS

ID	Requirement
FR6	Full drag/resize/reposition with smart alignment guides & snapping
FR7	Swap layout per slide without losing its content
FR8	Global re-theme propagated via design tokens in one action
FR9	Manual element library: shapes, icons, charts, tables, embeds
FR10	Undo/redo and version history

7.3 COLLABORATION & SHARING

ID	Requirement
FR11	Shareable view-only web link with view analytics
FR12	Comment threads on slides v1.1
FR13	Export to PPTX, PDF, PNG/JPEG

7.4 CONTENT & IMAGE QUALITY

ID	Requirement
FR14	Enforced content density limits (concise / balanced / detailed)
FR15	Consistent visual style enforcement across the deck
FR16	Dedicated image-brief step: concrete subject extraction, decorative-vs-informational routing
FR17	Vision-model relevance check before an image is placed; fallback on failure
FR18	Bespoke technical/structural content routes to Path B: structured brief sent to a text-rendering image model per benchmark in 6.5.2

8. Non-Functional Requirements

Requirement	Target
Full deck generation	≤ 60s (p95) for 15 slides
Single-slide regeneration	≤ 10s
Layout QA catch rate	> 95% of overflow/overlap/contrast issues caught pre-render
Renderer consistency	Pixel-identical between generation & manual editor
Accessibility	WCAG AA contrast by default

9. Success Metrics

Metric	Target
Generated decks needing zero manual layout fixes	≥ 60% at launch, ≥ 80% by v2
Time from prompt to shareable deck	< 90s median
User rating of visual quality	≥ 4.2 / 5
Free-to-paid conversion (export/edit tier)	Baseline TBD vs. market benchmarks
Deck completion rate	≥ 70%

10. Phased Roadmap

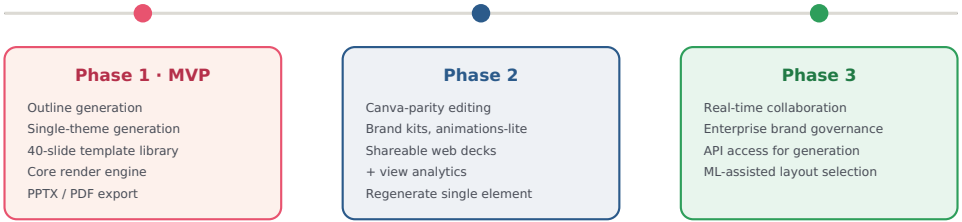


Fig. 4 — Delivery phases

11. Risks & Open Questions

- **Design-system investment is front-loaded:** 40–60 professionally designed templates is a real design-team commitment before AI work pays off — this is primarily a design-systems project, not a prompt-engineering one.
- **Differentiation risk:** matching Gamma's quality without a distinct wedge (speed, price, integration, vertical focus) means competing on parity rather than advantage.
- **Licensing:** stock imagery, fonts, and icon sets need clear commercial licenses at scale.
- **Open question:** does layout selection stay rule-based as the template library grows, or does it eventually need a dedicated ML model? Revisit after Phase 1 usage data.

12. Technical Implementation Plan — Athena VI Integration

This feature ships inside the existing **Athena VI** backend (`AthenaVI_backend`) alongside the current avatar-video editor loop, under a shared "Virtual Studio" umbrella — reusing existing infra rather than standing up a parallel system.

12.1 Integration Model (Recommendation)

Chosen approach: shared core infra, separate editor UI, embeddable later. `Project` gains a `type` discriminator (`VIDEO` | `PRESENTATION`) so a deck is still a `Project` — inheriting folder placement, comments, storage accounting, workspace scoping — without a data-model migration later if avatar-video scenes need to embed as slide elements.

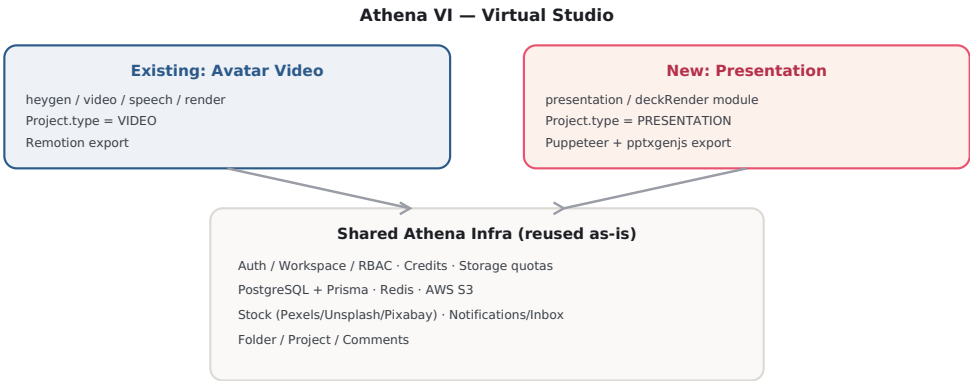


Fig. 7 — Shared infra vs. new modules

12.2 Tech Stack Mapping

Need	Existing Athena piece reused	New addition
API framework, validation, auth	Express 5, Joi, JWT/refresh, RBAC	New <code>presentation</code> module, same 4-layer pattern
System of record	PostgreSQL + Prisma 7	<code>Deck</code> , <code>Slide</code> , <code>DeckTemplate</code> , <code>SlideGenerationJob</code> , <code>DeckExport</code>
Caching / rate limits	Redis	Generation rate limits, moderation caching
Asset storage	AWS S3	New prefix convention for deck assets/exports
Stock imagery (Path A)	Existing <code>stock</code> module	None — reused directly
AI job pattern	heygen/video/speech create→poll→S3-sync→charge pattern	Same pattern for content/image/diagram jobs
Server-side composition	Remotion (architectural precedent)	New PPTX/PDF render module (12.7)
Credits & billing	<code>credit</code> module, <code>creditPricing.js</code>	New line items (12.10)
Storage quota	<code>storage</code> module	Reused unchanged

12.3 Data Model Additions (Prisma)

```
model Deck {
  id          String    @id @default(cuid())
  projectId   String    @unique
  project     Project    @relation(fields: [projectId], references: [id])
  themeTokens Json       // palette, font pairing, spacing, image style
  outline     Json       // editable pre-generation outline
  status      String    // DRAFT | GENERATING | READY | FAILED
}

model Slide {
  id          String    @id @default(cuid())
  deckId      String
  order       Int
  contentType String    // title | bullet_list | comparison | stat | chart ...
  layoutId    String    // DeckTemplate id
  content     Json
  imageRef    Json?     // { source: stock|ai_gen|path_b, assetId, brief }
  status      String    // PENDING | GENERATING | READY | FAILED
}

model DeckTemplate {
  id          String    @id @default(cuid())
```

```
  contentType String
  variant      String
  schema      Json    // grid/slot definition, see 6.3
  isActive    Boolean @default(true)
}

model SlideGenerationJob {
  id          String @id @default(cuid())
  slideId     String
  jobType     String // CONTENT | IMAGE_PATH_A | IMAGE_PATH_B | VALIDATION
  status      String // PENDING | RUNNING | SUCCEEDED | FAILED
  requestHash String // idempotency key, same as HeygenResponse
  creditCharged Boolean @default(false)
}

model DeckExport {
  id          String @id @default(cuid())
  deckId      String
  format      String // PPTX | PDF
  status      String // QUEUED | RENDERING | READY | FAILED
  s3Key       String?
  creditCharged Boolean @default(false)
}
```

12.4 API Contracts (new presentation module)

Method & Path	Purpose
POST /:workspaceId/projects (type=PRESENTATION)	Create a deck project
POST .../decks/outline	Generate editable outline from prompt/doc
PATCH .../decks/outline	User edits outline pre-generation
POST .../decks/generate	Kick off full generation
GET .../decks/status	Poll per-slide generation status
POST .../decks/slides/:slideId/regenerate	Regenerate one slide's content or image
PATCH .../decks/slides/:slideId	Manual edit save
POST .../decks/theme	Set/update deck-wide theme tokens
POST .../decks/export	Queue PPTX/PDF export
GET .../decks/export/:exportId	Poll export status, presigned download URL

All routes use the existing `successResponse` / `errorResponse` envelope, `authMiddleware` , `requireWorkspaceRole` , and Joi `validate` middleware — matching current conventions exactly.

12.5 Async Generation Flow

Mirrors the proven HeyGen video pattern — no new job-queue infra needed. Client calls generate → credit pre-check → `Deck` / `Slide` rows created → jobs run per slide with idempotency hashing → client polls status and slides render progressively as each completes → credits charged on success only, matching today's charge-on-success pattern.

12.6 Failure & Edge Cases

Case	Handling
LLM call times out/errors	Retry once w/ backoff; fallback to safe template + inbox notification
Content doesn't fit any template	Fallback to generic variant, stronger truncation
Image gen/search returns nothing	Theme-colored placeholder; slide still usable
Path B fails relevance/text check	Retry stricter prompt, then fallback to Path A
Unparseable source document	4xx error before any credit spend
Moderation flag	Block generation, no credit charge, actionable error
Regenerate after manual edits	Manually-edited slides preserved unless explicit overwrite confirmed
Partial deck failure	Deck marked READY with partial:true; failed slides flagged individually

12.7 Export (PPTX/PDF) Fidelity Spec

New `deckRender` module, parallel to the existing Remotion `render` module. Headless Chromium (Puppeteer) renders each slide from the same schema+tokens used for in-app preview (WYSIWYG guarantee). `pptxgenjs` maps schema into native, editable PowerPoint objects (not

flattened images) — Path B slides embed as a single image, per their nature. PDF uses Puppeteer print-to-PDF from the same rendered HTML. A `SlideRenderCache` (equivalent to `SceneRenderCache`) avoids re-rendering unchanged slides. Job flow, credit charging, and notification pattern match the existing Remotion export UX exactly.

12.8 Content Moderation & Safety

Prompt-level moderation check before any generation spend (no credits charged on block); generated slide copy and images pass a moderation/relevance pass before being shown (reusing the vision-model check from 6.5.1); Path B prompts are built entirely from structured, upstream-approved content, inherently limiting open-ended unsafe generation.

12.9 Template Production Workflow

Video-scene templates don't exist yet either — both are being built together as one system, not staggered. They share the same lifecycle (create, version, preview, activate/deactivate, RBAC) even though their content schemas are entirely different domains, so this is built as **one generic template system with a type discriminator**, following the same pattern as `Project.type` .

Layer	Shared across both types?
Storage, versioning, isActive, RBAC, audit log	Yes — one generic <code>Template</code> model and admin module
Superadmin CRUD routes/UI shell (list, filter by type, create, deactivate)	Yes — one "Templates" section with a type filter
Schema shape	No — video (canvas/timeline/avatar placement) vs. deck (12-col grid + slots) are different domains
Live preview renderer	No — video uses Remotion scene preview; decks use the render engine (6.7)

Revised Prisma model (replaces the deck-only `DeckTemplate` from 12.3 with a generic model):

```
enum TemplateType {
  VIDEO_SCENE
  DECK_LAYOUT
}

model Template {
  id      String      @id @default(cuid())
  type    TemplateType
  name    String
  contentType String?  // deck-only: title | bullet_list | comparison ...
  variant String?
  schema  Json         // shape depends on `type` - validated per-type
  version Int          @default(1)
  isActive Boolean      @default(true)
  createdBy String      // superadmin user id
}
```

`Slide.layoutId` (12.3) references `Template.id` where `type = DECK_LAYOUT` .

12.9.1 AUTHORING WORKFLOW (OPTION B, REFINED)

Raw JSON editing plus a manual "preview" button is a bad loop in practice — you can't correctly guess grid coordinates blind, so it becomes guess → preview → wrong → adjust → preview again. The fix isn't a full free-form drag-and-drop canvas builder (a much bigger, separate build); it's narrower: **only slot placement needs to be visual**, everything else is fine as form fields.

Field	Needs to be visual?	Editor
name, contentType, variant	No	Text/dropdown fields
Per-slot properties (fit, max_lines, max_words)	No	Number/dropdown fields
<code>region</code> (grid coordinates)	Yes — can't be authored correctly blind	Visual grid picker

Step by step (deck layout templates):

1. Open Superadmin → Templates → New Template, select type `DECK_LAYOUT` .
2. Fill metadata fields (name, content type, variant) — plain form, no JSON.
3. Place slots on a **clickable 12-column grid overlay** — click-and-drag to draw each slot's region directly on the grid; the tool writes the `region` value, no hand-typed coordinates.
4. **Live preview updates automatically as you drag** (debounced against the real render engine, 6.7), with sample content flowing into the slot — overlaps and bad sizing are visible immediately.
5. Set per-slot properties via form fields next to each placed slot.
5. Save as draft (`isActive: false`) — doesn't affect the live layout selector.
7. Publish/activate — flips `isActive: true` , eligible immediately, no deploy.
3. Editing later creates a new version rather than mutating in place, so decks already using v2 don't silently change when v3 publishes.
3. A raw-JSON escape hatch remains available for rare constraints the form doesn't cover, but it's not the primary path.

Video-scene templates follow the same lifecycle (steps 1-2, 6-8 identical), but step 3's visual placement is a canvas/timeline editor against the Remotion scene preview instead of a grid — the domain-specific piece — while metadata, versioning, and publish flow are the same shared machinery.

Who does this work: a design-technologist or engineer comfortable with structured layout tools, not a fully non-technical operator — that's the real gap between this (Option B+) and a true no-code visual builder (Option C), which remains out of v1 scope.

12.10 Credits & Billing Mapping

HeyGen billing is deterministic (duration/quality tier known before the job runs), fitting a flat pre-check→charge-fixed-amount pattern. OpenAI billing (tokens for content, resolution/quality for images) is usage-based and variable per call — it needs one genuinely new mechanic, not just new numbers in the same table.

Pattern	When Used	Mechanics
Flat estimated charge	Per-slide content, Path A/B images, export	Content-density caps (FR14) bound max tokens per call → known ceiling → priced like HeyGen, no reconciliation needed
Estimate → reconcile	Outline generation only	Open-ended length → pre-check against estimated band (402 if insufficient) → charge actual usage-based amount once token counts return

Action	Cost Basis	AC Charging Pattern
Outline generation	GPT input+output tokens	Estimate at pre-check, charge actual on success
Per-slide content generation	GPT tokens, bounded by FR14 caps	Flat estimated charge per slide
Path A image (AI-gen)	OpenAI image pricing tier	Flat AC cost per image
Path B composite diagram	OpenAI image pricing tier, higher tier	Flat AC cost, higher than Path A
Single-slide regeneration	Same basis as initial generation	Same charge as initial generation
PPTX/PDF export	Compute only, no OpenAI call	Flat AC cost, unchanged from Remotion pattern

Conversion formula — same shape as the existing HeyGen PAYG-to-AC conversion, with a second lookup table keyed to OpenAI's per-token/per-image pricing:

AC_cost = ceil((openai_usage_cost_usd * margin_multiplier) / AC_to_usd_rate)

```
// creditPricing.js - new OpenAI line items alongside existing HeyGen tables
const OPENAI_PRICING = {
  outline: {
    basis: 'usage', // estimate -> reconcile
    inputTokenRateUsd: 0.000003,
    outputTokenRateUsd: 0.000015,
    estimateTokens: { input: 800, output: 1200 },
  },
  slideContent: {
    basis: 'flat', // bounded by FR14 caps
    maxTokens: { input: 400, output: 300 },
    flatAcCost: 2,
  },
  imagePathA: { basis: 'flat', resolutionTier: 'standard', flatAcCost: 4 },
  imagePathB: { basis: 'flat', resolutionTier: 'hd', flatAcCost: 9 },
  deckExport: { basis: 'flat', flatAcCost: 3 },
};

function toAcCost(usageCostUsd, marginMultiplier = 1.3) {
  return Math.ceil((usageCostUsd * marginMultiplier) / AC_TO_USD_RATE);
}

// Outline: pre-check against estimate, reconcile against actual usage
// returned in the OpenAI response (prompt_tokens / completion_tokens).
function reconcileOutlineCharge(usage) {
  const usdCost =
    usage.prompt_tokens * OPENAI_PRICING.outline.inputTokenRateUsd +
    usage.completion_tokens * OPENAI_PRICING.outline.outputTokenRateUsd;
  return toAcCost(usdCost);
}
```

This keeps the existing charge-on-success discipline (failed jobs never bill) while adding the one genuinely new mechanic Athena's pricing layer doesn't have yet: usage-based reconciliation for the single action where cost isn't boundable up front.

12.11 Acceptance Criteria (Sample)

FR	Acceptance Criteria

FR4	p95 time from generate call to deck READY ≤ 60s for 15 slides, 10 concurrent decks in staging
FR5	Full-deck regenerate leaves manuallyEdited slides unchanged unless explicit overwrite confirmed
FR16	Chart/comparison content types never call the Path-A image source
FR18	Content with no matching template routes to Path B, not stock search

12.12 QA / Testing Strategy

Automated layout QA doubles as a CI regression gate for template + content combinations; visual regression via Puppeteer screenshot snapshots per template variant; weekly human eval loop against the 4.2/5 visual-quality target; export fidelity tests round-trip exported PPTX files programmatically to confirm text/position/image accuracy.